

F E I

Tentamen i Programmeringsteknik 2000-03-06, kl 8.00-13.00

Anvisningar

Hjälpmedel: Holm, Objektorienterad programmering och Java, samt Java-snabbreferens.

Skrivningen innehåller fyra uppgifter. Du bör behandla samtliga uppgifter.

Obs. Senast 3:e arbetsdagen efter tentamen anslås på anslagstavlan vilka som deltagit i tentamen men som, enligt institutionens noteringar, inte uppfyller tentamensvillkoren (alla 18 övningarna och de 3 inlämningsuppgifterna godkända). Deras skrivningar annulleras den 6:e arbetsdagen efter tentamen, såvida inte rättelse skett innan dess eller ansvarig lärare lämnat dispens.

Närmaste tentamenstillfällen i kursen:

Onsdag 24 maj 2000, kl 8-13

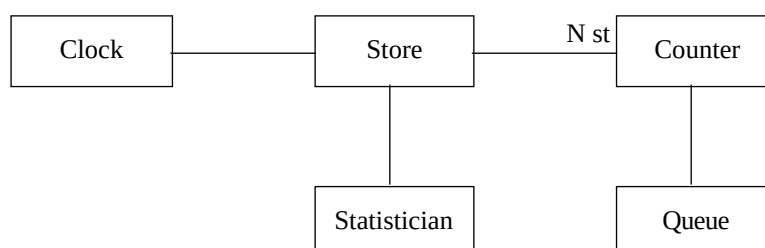
Fredag 25 augusti 2000, kl 8-13

Något om bedömningen

- Tänk på att du med dina lösningar vänder dig till en mänsklig rättare, inte till en dator.
- Du kan förutsätta att rättaren är väl insatt i uppgifterna.
- Rättaren fäster ingen större uppmärksamhet vid fel som en kompilator enkelt skulle upptäcka.
- Det är viktigt att du gör lämpliga och tydliga indragningar och att du använder vettiga variabelnamn så att din lösning går lätt att följa. En god handstil är ett plus.
- Tvekar du mellan att presentera en smart lösning eller en lösning som är enkel att följa, välj den enklare versionen.
- När det gäller betyget 3 så har det ingen betydelse om någon lösning är onödigt ineffektiv.

Problemet

Samtliga fyra uppgifter ingår i ett program där man testar kösystemet i en butik (tillhörig klassen *Store*). I butiken finns ett antal kassor (tillhöriga klassen *Counter*). Till butiken hör en klocka (tillhörig klassen *Clock*). Man vill beräkna hur länge butikens kunder behöver vänta innan de kommer fram till kassan. Beräkningen görs av en statistiker (tillhörig klassen *Statistician*). Till varje kassa hör en kö (tillhörig klassen *Queue*). Kunderna väljer alltid den kortaste kön.



Uppgift 1

Klasserna Clock och Statistician har följande uppbyggnad:

```
class Clock {
    ...    // inför lämpliga attribut!

    /** skapa en klocka */
    public Clock() {
    }

    /** nollställ klockan */
    public void reset() {
        ...    // implementera!
    }

    /** stega fram klockan en minut */
    public void tick() {
        ...    // implementera!
    }

    /** tag reda på vad klockan visar */
    public int getTime() {
        ...    // implementera!
    }
}

class Statistician {
    ...    // inför lämpliga attribut!

    /** skapa en statistiker */
    public Statistician() {
    }

    /** gör statistikern redo att göra beräkningar, dvs gör nödvändiga
        nollställningar av attributen */
    public void reset() {
        ...    // implementera!
    }

    /** tag hand om ("notera") ett nytt mätvärde x (en väntetid) */
    public void addSample(int x) {
        ...    // implementera!
    }

    /** tag reda på antalet noterade mätvärden */
    public int getNbr() {
        ...    // implementera!
    }

    /** tag reda på medelvärdet av de noterade mätvärdena */
    public double getAverage() {
        ...    // implementera!
    }

    /** tag reda på standardavvikelsen för de noterade mätvärdena */
    public int getStdDev() {
        ...    // implementera!
    }
}
```

Bestäm lämpliga attribut för klasserna Clock och Statistician och implementera samtliga operationer i de två klasserna.

Ledning. Standardavvikelsen $s = \sqrt{\sum x_i^2 - nm^2 / (n-1)}$. Här är x_i ett mätvärde, n antalet mätvärden och m medelvärdet av mätvärdena.

Uppgift 2

Klassen Counter som beskriver en kassa i en butik har följande uppbyggnad:

```
class Counter {
    private Store store;           // kassan finns i butiken store
    private Queue q;               // kön till denna kassa
    private int freeTime;          // tidpunkt när kassan är ledig nästa gång

    /** skapa en kassa i butiken store, kassans kö är tom */
    public Counter(Store store) {
        ... // implementera!
    }

    /** tag reda på antalet personer som finns i kassans kö,
        personerna i kön representeras av sina ankomsttider */
    public int queueLength() {
        ... // implementera!
    }

    /** lägg in en ny person (personens ankomsttid time) sist i kön */
    public void intoQueue(int time) {
        ... // implementera!
    }

    /** tag reda på om kassan är ledig just nu */
    public boolean free() {
        ... // implementera!
    }

    /** tag reda på aktuell väntetid för den första personen i kön */
    public int waitingTimeForFirst() {
        ... // implementera!
    }

    /** betjäna den första personen i kön, timeToServe är betjäningstiden */
    public void serveFirst(int timeToServe) {
        ... // implementera!
    }
}
```

Implementera konstruktorn och de fem operationerna i klassen Counter. Klassen Store är beskriven i uppgift 3. Klassen Queue är beskriven i uppgift 4.

Ledning. I butiken store kan man ta reda på aktuell tid med:

```
store.getClockTime()
```

Uppgift 3

Klassen Store har följande uppbyggnad:

```
class Store {
    private Counter[] counter;    // vektor för kassorna
    private int nbrOfCounters;    // antal kassor
    private Clock clock;          // butikens klocka

    /** skapa en butik med nbrOfCounters kassor */
    public Store(int nbrOfCounters) {
        ... // implementera!
    }

    /** tag reda på vad butikens klocka visar */
    public int getClockTime() {
        ... // implementera!
    }

    /** tag reda på den kassa som har kortaste kön,
        denna operation används i operationen simulate */
    private Counter findShortest() {
        ... // implementera!
    }

    /** simulera hur kunder kommer och betjänas från tiden 0 till nbrMinutes,
        hur simuleringen går till beskrivs i algoritmen nedan */
    public void simulate(int nbrMinutes, double mv, int maxServingTime) {
        ... // implementera!
    }
}
```

Algoritm för operationen simulate:

givet:

nbrOfCounters kassor
en kö till varje kassa

för varje minut från 0 till nbrMinutes:

s kunder anländer

varje kund ställer sig i den *kortaste* kön

för varje kassa:

om kassan är ledig och dess kö ej är tom så

notera hur länge den första kunden i kön har väntat

börja betjäna den första kunden i kön

notera att kassan blir ledig om m minuter (=betjäningstiden)

skriv ut antalet betjänade kunder, medelvärde och standardavvikelse för deras väntetider (betjäningstiderna inräknas ej)

beräkna och skriv också ut antalet kunder som är kvar i köerna efter tiden nbrMinutes

s är ett Poisson-fördelat slumpstal med medelvärde mv.

m är ett slumpmässigt heltal, rektangelfördelat i intervallet 1..maxServingTime.

Om rng är en RandomNumberGenerator erhåller man s och m med satserna:

```
int s = rng.randPoisson(mv);
int m = rng.randInt(1,maxServingTime);
```

Implementera konstruktorn och de tre operationerna i klassen Store. Skapa en Statistician som sköter om de statistiska beräkningarna!

Uppgift 4

För hantering av köerna till kassorna finns en klass Queue med följande specifikation:

```
specification Queue {  
    /** skapa en kö där man kan lägga in tider (int-värden) */  
    Queue();  
  
    /** tag reda på antalet element (tider) i kön */  
    int getLength();  
  
    /** tag reda på tiden för den första i kön */  
    int getFirst();  
  
    /** tag bort det första elementet (tiden) från kön */  
    void removeFirst();  
  
    /** lägg in ett nytt element med tiden time sist i kön */  
    void putInLast(int time);  
}
```

Implementera kön med användning av jlist-paketet:

```
class IntElement extends Link {...}  
  
class Queue extends Head {...}
```